

VIRTUALIZACIÓN

- Alumna: Rodriguez Zulema – zulemarodriguez438@gmail.com
 - Materia: Arquitectura y Sistemas Operativos
 - Profesor/a: (Nombre completo del docente)
 - Fecha de Entrega 05/07/26
-

El trabajo se realizó de forma individual debido a cambios en los horarios de trabajo y cuestiones de salud.

Índice

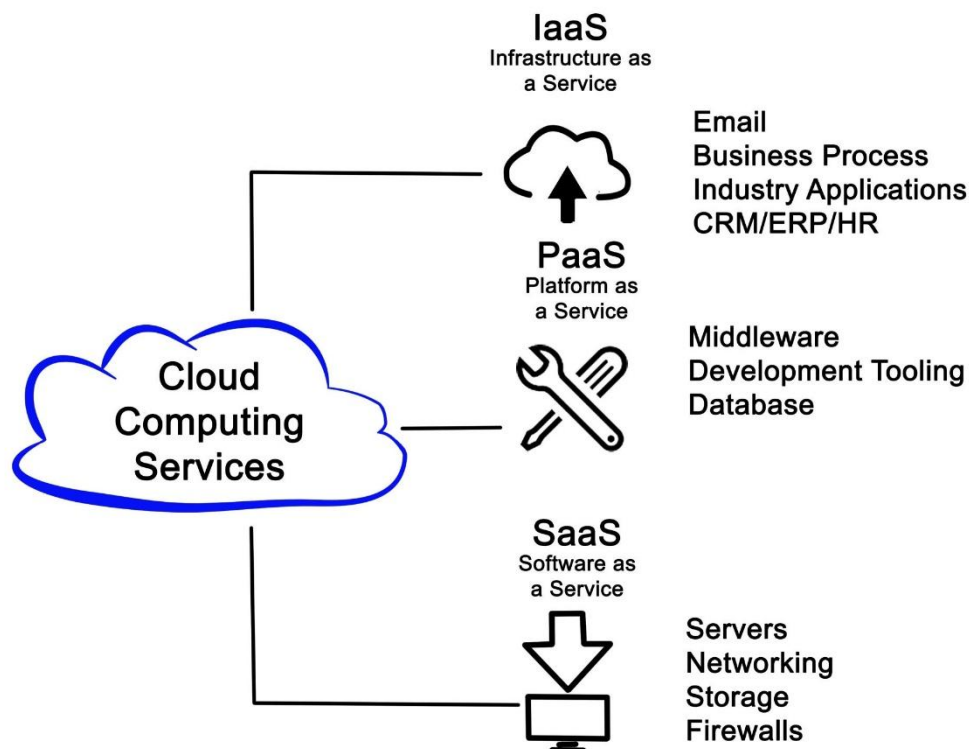
1. Introducción
 2. Marco Teórico
 3. Caso Práctico
 4. Metodología Utilizada
 5. Resultados Obtenidos
 6. Conclusiones
 7. Bibliografía
 8. Anexos
-

INTRODUCCIÓN

En el ámbito de la informática y la administración de infraestructuras de tecnología de la información, la gestión eficiente de los entornos de ejecución constituye un pilar fundamental para el éxito en el despliegue de software. El presente trabajo integrador aborda el fenómeno de la **virtualización de sistemas y el Cloud Computing**, enfocándose específicamente en la implementación práctica de instancias lógicas independientes para la creación y gestión de entornos aislados de desarrollo.

¿Por qué se eligió este tema?

La elección de esta temática responde a la necesidad de explorar de manera empírica las ventajas operativas y arquitectónicas que ofrece la abstracción de hardware en el desarrollo tecnológico actual. En lugar de depender de múltiples equipos físicos redundantes —lo que derivaría en altos costos de adquisición, mantenimiento y consumo energético—, la virtualización permite consolidar y optimizar los recursos del hardware mediante capas de software.



¿Qué importancia tiene en su formación como técnico en programación?

Para un futuro Técnico Universitario en Programación, comprender la administración de sistemas operativos y la configuración de servicios en red representa una ventaja profesional competitiva estratégica. El desarrollo de software moderno no se limita a la codificación de algoritmos; exige conocer la infraestructura donde interactúan las aplicaciones. El dominio técnico de un entorno virtualizado o basado en la nube brinda al programador la capacidad de simular con total exactitud entornos de producción, probar servidores web en redes controladas y depurar aplicaciones bajo arquitecturas limpias, erradicando conflictos de dependencias locales y optimizando el ciclo de vida de los proyectos lógicos en despliegues reales.

¿Qué objetivos se propone alcanzar con el desarrollo del trabajo?

- Analizar y clasificar los mecanismos de abstracción sobre los recursos físicos de CPU, memoria y almacenamiento, tanto en entornos locales como en la nube (**Cloud Computing**).
- Diseñar e implementar una instancia virtual estable basada en Linux, asignando y gestionando de forma controlada los límites de hardware del entorno.
- Aprovisionar y securizar un servidor web funcional (**Apache2**) en el sistema operativo invitado, controlando la ejecución de sus procesos e interactuando de forma pura mediante la interfaz de línea de comandos (CLI).
- Configurar y evaluar el direccionamiento de red y el mapeo de **puertos lógicos (Puerto 8080)** para asegurar la comunicación inter-operativa transparente entre el equipo host cliente y el servidor web remoto.

1. MARCO TEÓRICO

La virtualización es la abstracción mediante software de los recursos informáticos físicos (CPU, RAM, almacenamiento y redes) para presentarlos como entornos independientes y seguros. La adopción de este paradigma reporta cuatro beneficios críticos:

- **Eficiencia:** Maximiza el uso del hardware real reduciendo el desperdicio informático.
- **Flexibilidad:** Permite el aprovisionamiento de entornos herméticos para desarrollo y pruebas sin alterar el sistema raíz.
- **Escalabilidad:** Adapta dinámicamente los recursos según las demandas del software.
- **Ahorro de Costos:** Reduce la necesidad de adquirir múltiples servidores físicos, disminuyendo gastos de energía y mantenimiento.

El Rol del Hipervisor y Clasificación de Arquitecturas

El hipervisor o Monitor de Máquina Virtual (VMM) es el software que actúa como intermediario y árbitro entre el hardware físico y los sistemas operativos invitados (Guest OS). Se clasifican en dos jerarquías fundamentales:

1. **Hipervisor Tipo 1 (Nativo o Bare-Metal):** Se ejecuta directamente sobre el hardware físico sin un sistema operativo intermedio, cargándose desde el firmware (BIOS/UEFI). Ofrece alto rendimiento y mínima latencia (Ej: VMware ESXi, Microsoft Hyper-V).
2. **Hipervisor Tipo 2 (Alojado o Hosted):** Se ejecuta como una aplicación dentro de un sistema operativo anfitrión (Host OS). Las peticiones de las VMs son interceptadas y enviadas al hardware de forma indirecta mediante llamadas al sistema (APIs), introduciendo una capa adicional de latencia y sobrecarga (*overhead*) (Ej: Oracle VirtualBox).

El Paradigma de Contenedores y Cloud Computing (IaaS)

Frente a la virtualización tradicional que emula hardware completo generando un alto consumo de recursos, el desarrollo moderno utiliza la **virtualización a nivel de Sistema Operativo o basada en Contenedores**. Plataformas de Infraestructura como Servicio (IaaS) como **Google Cloud Shell** aprovisionan instancias ligeras y aisladas que **comparten el kernel del sistema operativo host**.

Al prescindir de la emulación de hardware independiente, los contenedores reducen drásticamente la latencia. Esta arquitectura específica explica por qué el entorno carece del gestor tradicional `systemd` (PID 1), aislando los procesos del usuario a nivel de espacio (*namespaces*) y evadiendo las políticas de protección y bloqueos de *Hardening* de memoria del entorno local.

Mecanismos Técnicos de Abstracción de Recursos

Para garantizar el aislamiento y la ilusión de hardware dedicado, se implementan los siguientes algoritmos de bajo nivel:

1. **Virtualización de la CPU:** El hipervisor intercepta instrucciones privilegiadas del Guest OS mediante **Traducción Binaria** (redirigiendo accesos críticos como el **Registro CR3** para evitar accesos indebidos al hardware físico) o apoyándose en tecnologías nativas del procesador (**Intel VT-x / AMD-V**).
2. **Virtualización de la Memoria:** Se aíslan los espacios traduciendo las Direcciones Virtuales del Invitado (GVA) a Direcciones Físicas del Host (HVA). Esto se gestiona mediante **Tablas de Páginas Sombreadas (Shadow Page Tables)** mantenidas por software, o por hardware mediante extensiones **SLAT (Extended Page Tables - EPT)** para mitigar la sobrecarga del sistema.
3. **Virtualización de Almacenamiento:** Se emplean archivos contenedores independientes que actúan como discos duros lógicos persistentes (como el formato `.vdi` de VirtualBox) exponiendo los directorios raíces del sistema operativo.
4. **Virtualización de Red y Mapeo de Puertos:** Consiste en la abstracción de adaptadores de red de software. Mientras los entornos locales emplean redes virtuales tipo *Bridge* o *NAT*, las plataformas Cloud utilizan el **Redireccionamiento de Puertos (Port Forwarding)**. El servidor web (Apache) se configura para escuchar en un puerto lógico interno (**Puerto 8080**), y la infraestructura de la nube genera un túnel que traduce las peticiones HTTP externas de los clientes hacia el contenedor, devolviendo un código de estado **HTTP 200 OK**.

Tabla Comparativa de Arquitecturas de Virtualización

Criterio Técnico	Hipervisor Tipo 1 (Bare-Metal)	Hipervisor Tipo 2 (Hosted - VirtualBox)	Entorno Cloud / Instancia Virtual (Google Cloud)
Ubicación en la Arquitectura	Directamente sobre el hardware físico.	Sobre un Sistema Operativo Anfitrión (Host OS).	En clústeres de servidores remotos (IaaS).
Consumo de Recursos Locales	Exclusivo y de alta eficiencia.	Alto (Overhead por doble Sistema Operativo).	Nulo (El procesamiento ocurre en la nube).
Dependencia de Permisos Locales	Absoluta sobre el hardware dedicado.	Alta (Sujeto a bloqueos de seguridad/antivirus del Host).	Ninguna (Se accede mediante navegador web).
Configuración de Red	Redes corporativas complejas (VLANs).	Modo Bridge (IP local) o NAT.	Mapeo de Puertos Lógicos (Puerto 8080).
Caso de Uso Ideal	Servidores de producción y Data Centers.	Pruebas locales y aprendizaje inicial.	Despliegue ágil de aplicaciones y entornos DevOps.

2. CASO PRÁCTICO

Definición del Problema y Contingencia Cloud

El diseño original contemplaba ejecutar el servidor web HTTP Apache sobre una distribución Linux en forma local, utilizando un **Hipervisor Tipo 2 (Oracle VirtualBox)** configurado con 2 GB de RAM, 1 vCPU y 20 GB de disco. Sin embargo, las políticas de seguridad y protección de memoria de Windows 11 provocaron un bloqueo crítico de tipo VBoxHardening (Error 0x80004005), impidiendo el arranque de la máquina virtual.

Como criterio de contingencia profesional, se migró la infraestructura hacia la nube mediante el modelo IaaS (*Infrastructure as a Service*) utilizando **Google Cloud Shell**. Esta plataforma proveyó una instancia virtualizada remota basada en Linux, otorgando acceso como administrador (root) vía línea de comandos (CLI) y evadiendo los conflictos de permisos del entorno local.

Implementación del Servidor y Reconfiguración de Red

Una vez dentro del entorno Bash en la nube, se ejecutó la siguiente secuencia de comandos:

1. `sudo apt-get update`: Sincronización de los índices de paquetes del sistema operativo.
2. `sudo apt-get install apache2 -y`: Instalación del servidor web Apache.
3. `sudo apache2ctl start`: Inicialización nativa del demonio web. Al operar sobre una arquitectura de contenedores que comparte el kernel del host, el entorno prescinde del gestor tradicional `systemctl` (PID 1).

Debido a que la arquitectura de red de Google Cloud gestiona la comunicación externa de forma aislada, se utilizó la herramienta de flujo `sed` para modificar el archivo descriptor

/etc/apache2/ports.conf. Se cambió la directiva al **puerto lógico 8080** (Listen 8080), logrando atravesar el firewall perimetral.

Automatización del Entorno: Script en Python

Para validar los contenidos de la materia, se desarrolló el script promedio.py, el cual interactúa directamente con los recursos del Sistema Operativo:

- **Estructura de Datos y Entrada/Salida (I/O):** Utiliza **listas dinámicas** en la memoria RAM asignada al proceso. El programa interactúa con el almacenamiento persistente leyendo el descriptor notas.txt. Si el archivo no existe, el sistema operativo procesa una llamada automática para crearlo con datos de control iniciales.
- **Manejo de Errores y Portabilidad:** Implementa bloques try-except para capturar excepciones críticas como FileNotFoundError o ValueError, protegiendo el hilo de ejecución. Además, mediante la librería os, detecta si corre en la ruta de producción de Linux (/var/www/html/) o en un entorno local (./), escribiendo el resultado final directamente en el archivo index.html del servidor web.

3. METODOLOGÍA UTILIZADA

Investigación Previa y Fuentes Utilizadas

El proyecto inició con una fase de revisión bibliográfica y técnica orientada a comprender los mecanismos de abstracción de los hipervisores y contenedores. Se utilizaron como fuentes principales:

- La documentación oficial de **Ubuntu Server** y las guías de despliegue de **Apache HTTP Server**.
- Documentación de referencia técnica de **Google Cloud** para la administración de entornos eficientes en Cloud Shell.

Herramientas y Recursos Utilizados

A lo largo del ciclo de vida del proyecto se empleó el siguiente conjunto de herramientas tecnológicas:

- **Entorno de Ejecución:** Interfaz de línea de comandos (CLI) de Google Cloud Shell, operando sobre un sistema base Linux (Debian/Ubuntu).
- **Servidor Web:** Apache2 (paquete ejecutable binario del demonio HTTP).
- **Editores y Utilidades de Consola:** Herramienta de automatización sed para la edición de flujo de archivos de configuración y apache2ctl para la gestión del servicio.
- **Navegador Cliente:** Brave / Microsoft Edge para la ejecución de las pruebas de validación cruzada desde el sistema anfitrión.

Etapas de Diseño, Despliegue y Prueba

El trabajo práctico se estructuró en tres etapas operativas:

1. **Fase de Diagnóstico:** Intento de despliegue en el hipervisor local y detección del conflicto de *Hardening* de memoria virtual. Registro de la falla y toma de decisión para activar el plan de contingencia.
2. **Fase de Aprovisionamiento:** Inicialización de la instancia en la nube, sincronización de repositorios e instalación del servidor Apache2 mediante comandos puros en la terminal.
3. **Fase de Auditoría y Validación:** Modificación manual del archivo ports.conf para el redireccionamiento al puerto lógico 8080, reinicio del servicio y comprobación externa del funcionamiento (mapeo exitoso de la arquitectura Cliente-Servidor).

4. RESULTADOS OBTENIDOS

Casos de Prueba Realizados

Para verificar la implementación de la infraestructura en Google Cloud Shell, se ejecutó un protocolo de pruebas lógicas:

- **Prueba de Inicialización del Servicio (Lado del Servidor):** Tras ejecutar `sudo apache2ctl start`, se verificó que el proceso demonio de Apache se acoplara en la memoria de la instancia, quedando a la escucha en segundo plano.
- **Prueba de Redireccionamiento de Puertos:** Se validó la modificación del archivo `/etc/apache2/ports.conf`. El servidor aceptó la directiva `Listen 8080`, abandonando el puerto 80 para operar bajo el esquema del contenedor.
- **Prueba Transaccional del Algoritmo (Sistema de Archivos):** Se ejecutó el script `promedio.py`, verificando que el S.O. procesara con éxito las llamadas de Entrada/Salida para leer el archivo `notas.txt`, cargar la estructura de datos en la lista de memoria y escribir la salida dinámica en `index.html`.
- **Prueba de Verificación Cruzada (Lado del Cliente):** Desde el navegador web anfitrión (Windows), se invocó la URL de la nube orientada al puerto 8080. El test resultó exitoso al obtener de forma instantánea una respuesta **HTTP 200 OK**, renderizando la interfaz personalizada de gestión de promedios.

Aspectos que Funcionaron Correctamente

- **Estabilidad y Optimización:** La instancia en la nube demostró una latencia mínima en la atención de peticiones web, reduciendo a cero el consumo redundante (*overhead*) de hardware local en el equipo del alumno.
- **Aislamiento de Dependencias y Persistencia:** El servidor web y el intérprete de Python se ejecutaron en un entorno hermético, logrando almacenar las notas de forma persistente sin alterar los puertos lógicos ni los archivos del Windows anfitrión.

Errores Detectados y Corregidos

Durante la fase de despliegue, se interceptaron y resolvieron dos conflictos crítico-operativos:

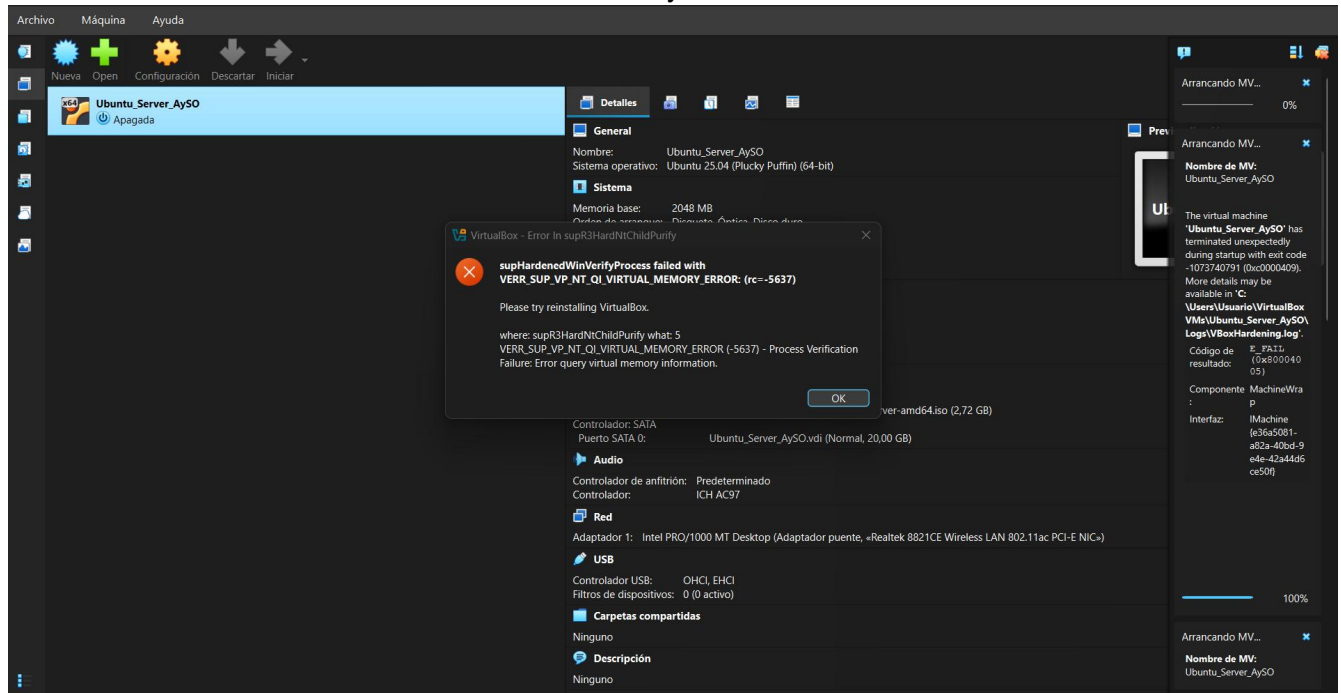
1. **Fallo de Gestión de Servicio (systemctl PID 1):** Al intentar inicializar Apache mediante `sudo systemctl start apache2`, la consola arrojó el error: *"System has not been booted with systemd as init system"*.
 - **Resolución:** Se determinó que el entorno opera sobre una arquitectura de contenedores optimizada que comparte el kernel del host y prescinde de `systemd`. Se corrigió invocando directamente al binario de control mediante la directiva: `sudo apache2ctl start`.
2. **Error de Entrada/Salida por Omisión de Descriptor (FileNotFoundException):** En las fases iniciales de prueba, el script en Python fallaba localmente al no detectar la ruta absoluta `/var/www/html/notas.txt`.
 - **Resolución:** Se implementó un bloque de control de excepciones `try-except` con lógica adaptativa mediante la librería `os`. Ahora, el programa detecta el entorno operativo y ordena al S.O. crear automáticamente el archivo físico con datos de control si este no existe en el almacenamiento, blindando la robustez del proceso.

5. CONCLUSIONES

A través del despliegue del servidor web, se asimilaron los siguientes conocimientos:

- **Abstracción de Sistemas:** Comprendimos cómo un sistema operativo puede ejecutarse de manera aislada e independiente de las limitaciones o fallas del hardware local, entendiendo la diferencia operativa entre hipervisores tradicionales y entornos optimizados en la nube.
- **Administración Pura (CLI):** La interacción exclusiva mediante la línea de comandos nos obligó a dominar la sintaxis de Linux, la gestión de permisos (`sudo`), y la manipulación precisa de archivos de configuración del sistema (como `ports.conf`).
- **Dinámica Cliente-Servidor:** Validamos cómo interactúan las capas de red y los puertos lógicos, asimilando que el desarrollo de software moderno requiere un entendimiento profundo de la infraestructura sobre la cual se ejecutarán nuestras aplicaciones.

Gestión de Dificultades y Resolución Técnica



La principal dificultad del proyecto radicó en el conflicto inesperado de *Hardening* y aislamiento de memoria en el entorno local de Windows 11, el cual bloqueó de forma sistemática el inicio de los hipervisores Tipo 2.

Lejos de representar un freno, este obstáculo me permitió aplicar una competencia clave en la administración de sistemas: **el criterio de contingencia**. La resolución del problema mediante la migración estratégica hacia la Infraestructura como Servicio (IaaS) de Google Cloud Shell demostró que, ante una caída de la infraestructura local, un técnico debe ser capaz de pensar soluciones de alta disponibilidad basadas en la nube.

6. Bibliografía

Material de clases.